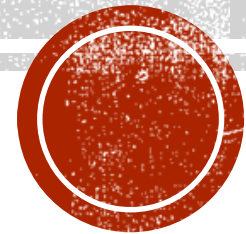


МОДЕЛИРОВАНИЕ КОЛЕБАНИЙ ПРУЖИННОГО МАЯТНИКА В QT

Курсовая работа по дисциплине
«Графика и компьютерные интерфейсы»

Выполнил:
Аксёнов Д.И.
АМ-24-09



Руководитель:
к.т.н. доцент
Прядко С.А.

Москва, 2026

ВВЕДЕНИЕ

Компьютерное моделирование физических процессов — важный инструмент современного образования и науки. Интерактивные приложения позволяют пользователю изменять параметры системы и сразу видеть отклик модели.

Классический объект для демонстрации механических колебаний — **пружинный маятник**: груз массой m на пружине жёсткостью k . При смещении возникает возвращающая сила, что приводит к гармоническим колебаниям.

Ключевые элементы работы: математическая модель, её численное решение, проектирование GUI с элементами управления и анимированной визуализацией.



ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

ЦЕЛЬ

Разработать интерактивное приложение на Python с использованием фреймворка Qt (PySide6), моделирующее и визуализирующее в реальном времени колебания пружинного маятника.

ЗАДАЧИ

- Построить **математическую модель** движения и преобразовать её к системе уравнений первого порядка.
- Реализовать **численное интегрирование** методом Эйлера с шагом Δt .
- Спроектировать **графический интерфейс**: анимация, регуляторы параметров, кнопки управления, график $x(t)$.
- Реализовать **интерполяционную отрисовку пружины** через кубические сплайны Catmull–Rom.
- Связать вычислительный модуль с модулем визуализации в едином приложении.

ФИЗИЧЕСКАЯ ПОСТАНОВКА

Гармонический осциллятор:

Груз массой m , закреплённый на пружине жёсткостью k , совершает колебания относительно положения равновесия под действием силы упругости.

Параметры моделирования:

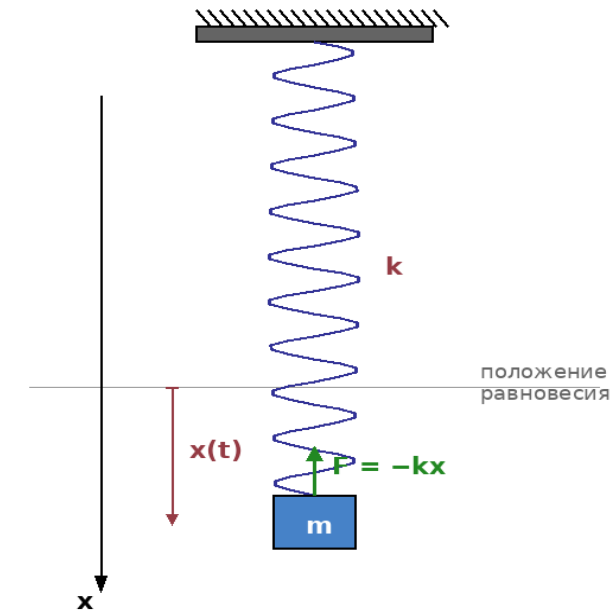
Масса груза m — определяет инертность системы. Больше масса — медленнее колебания (больше период).

Жёсткость пружины k — характеризует упругие свойства. Больше жёсткость — выше частота колебаний.

Трение в модели не учитывается — наблюдаются незатухающие колебания.

Период колебаний:

$$T = 2\pi \cdot \sqrt{(m/k)}$$



МАТЕМАТИЧЕСКАЯ МОДЕЛЬ

1. Второй закон Ньютона для пружинного маятника:

$$m \cdot x''(t) = -k \cdot x(t)$$

2. Переход к системе уравнений первого порядка:

$$\begin{aligned} dv/dt &= -(k/m) \cdot x \\ dx/dt &= v \end{aligned}$$

3. Аналитическое решение (для проверки):

$$\begin{aligned} x(t) &= A \cdot \cos(\omega \cdot t + \varphi), \\ \text{где } \omega &= \sqrt{k/m} \end{aligned}$$

Дифференциальное уравнение разбивается на пару уравнений для координаты и скорости — это позволяет применить пошаговое численное интегрирование.

ЧИСЛЕННОЕ РЕШЕНИЕ

Метод Эйлера

Простейший явный метод численного интегрирования: значения на следующем шаге вычисляются из значений на текущем.

Рекуррентные формулы:

$$\begin{aligned}a_n &= -(k/m) \cdot x_n \\v_n^{+1} &= v_n + a_n \cdot \Delta t \\x_n^{+1} &= x_n + v_n^{+1} \cdot \Delta t\end{aligned}$$

Параметры расчёта в программе:

- шаг $\Delta t = 0.02$ с (50 кадров в секунду);
- начальное смещение $x_0 = 0.10$ м, $v_0 = 0$.

```
update_simulation()

def update_simulation(self):
    if not self.is_running:
        return

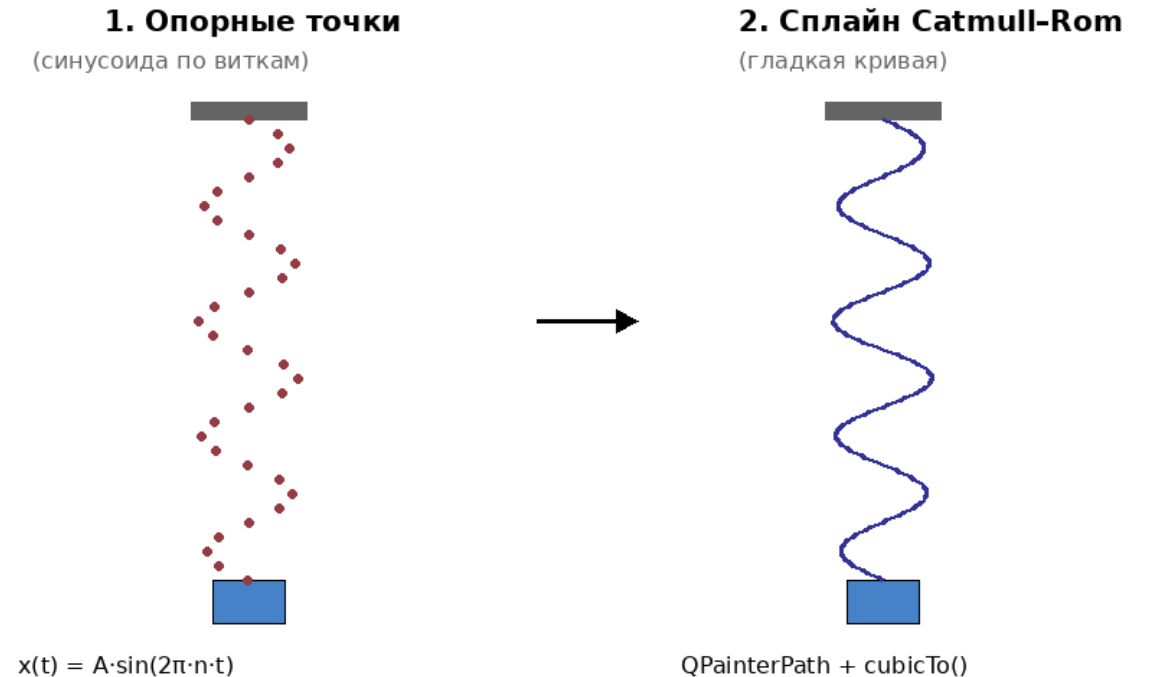
    # a = -(k/m) * x
    a = -(self.stiffness
          / self.mass) * self.x
    self.v += a * self.dt
    self.x += self.v * self.dt
    self.time += self.dt

    self.spring_widget.\
        set_physical_state(
            self.x, self.v)
    self.spring_widget.update()
```

ОТРИСОВКА ПРУЖИНЫ

Двухэтапная схема:

- Опорные точки — равномерно по длине пружины. Координата y растёт линейно, координата x задаётся синусоидой:
- Сплайн Catmull–Rom — для каждой пары соседних точек строится кубическая кривая Безье. Контрольные точки определяются по касательным.
- Класс `QPainterPath` и метод `cubicTo()` — отрисовка результата кистью толщиной 3 пикселя.



От набора опорных точек к гладкой кривой через кубические сплайны Catmull–Rom

$$A(t) = A^0 \cdot (1 - 0.2 \cdot |2t - 1|)$$

КЛАСС SpringPendulumWidget

Родительский класс: *QWidget*

Назначение: визуализация текущего состояния пружинного маятника. Отрисовка опоры, пружины и груза.

Ключевые методы:

set_physical_state(x, v) — приём текущего состояния из MainWindow.

get_mass_y_pixels() — перевод физических координат в пиксели.

draw_spring_interpolated() — построение опорных точек и сплайна.

paintEvent() — отрисовка всей сцены: опора, пружина, груз, подпись смещения.

```
paintEvent()

def paintEvent(self, event):
    p = QPainter(self)
    p.setRenderHint(
        QPainter.Antialiasing)

    # 1. Опора
    p.setBrush(QBrush(
        QColor(100,100,100)))
    p.drawRect(...)

    # 2. Пружина
    self.draw_spring_\
        interpolated(p)

    # 3. Груз
    mass_y = self.\
        get_mass_y_pixels()
    p.drawRect(...)
```

КЛАСС `MainWindow`

Родительский класс: `QMainWindow`

Назначение: управление симуляцией, хранение состояния, построение GUI и связь всех элементов управления с моделью.

Обязанности:

Физическая модель: переменные x , v , t и параметры m , k . Метод Эйлера в `update_simulation()`.

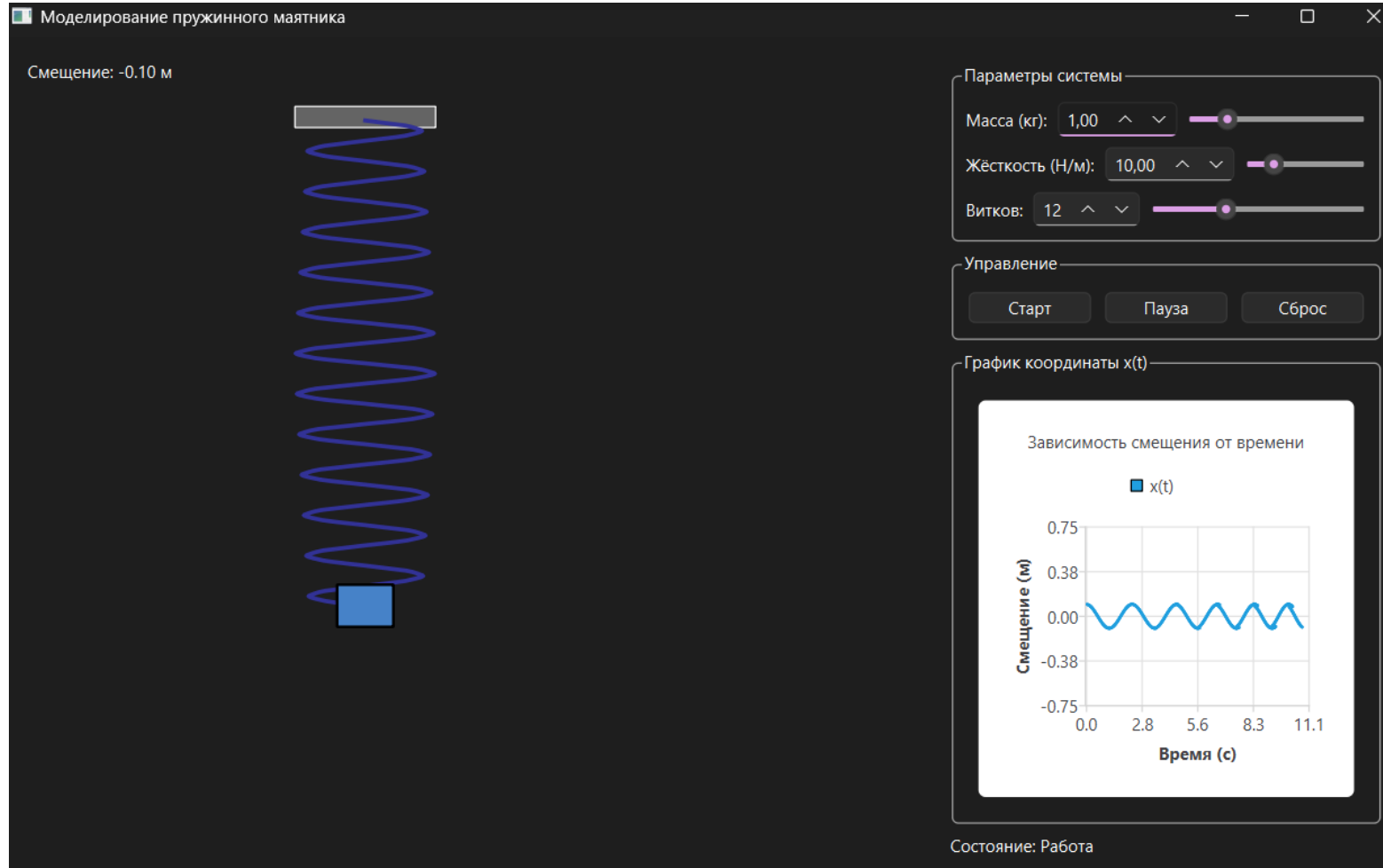
Анимация: `QTimer` с шагом $\Delta t = 20$ мс вызывает обновление симуляции и перерисовку виджета.

Конструирование GUI: `QHBoxLayout` делит окно на анимацию (слева) и панель управления (справа).

Двусторонняя связь: пары `QSlider` + `QDoubleSpinBox` синхронизируются через `blockSignals()`.

Графики: `QChart` + `QLineSeries` — точки $x(t)$ добавляются на каждом кадре, ось времени расширяется.

ИНТЕРФЕЙС ПРОГРАММЫ



ЭЛЕМЕНТЫ ОКНА

Область анимации

опора, пружина, груз

Параметры системы

масса, жёсткость, витки

Управление

Старт / Пауза / Сброс

График $x(t)$

обновляется в реальном времени

Главное окно при запущенной симуляции. Слева — анимация, справа — панель настроек и график.

ЗАКЛЮЧЕНИЕ

В ходе работы разработано интерактивное приложение моделирования пружинного маятника.

ФИЗИКА

Построена математическая модель гармонического осциллятора и реализовано её численное решение методом Эйлера с шагом 0.02 с.

ВИЗУАЛИЗАЦИЯ

Реализована плавная отрисовка пружины через интерполяцию кубическими сплайнами Catmull–Rom с переменной амплитудой витков.

ИНТЕРФЕЙС

Спроектирован GUI на PySide6: анимация в реальном времени, регулировка параметров, кнопки управления, динамический график $x(t)$.

Все задачи курсовой работы выполнены.

Список использованной литературы

1. Прохоренок Н. А. Python 3 и PyQt 6. Разработка приложений. / Н.А. Прохоренок, В.А. Дронов. - Санкт-Петербург : БХВ-Петербург, 2023. - 832 с.

2. Шелудько, В. М. Язык программирования высокого уровня Python. Функции, структуры данных, дополнительные модули: учебное пособие / В. М. Шелудько. – Ростов-на-Дону, Таганрог: Издательство Южного федерального университета, 2017. – 107 с.

3. Папилина Т.М., Прядко С.А., Фомочкина А.С. Применение модулей языка PYTHON в инженерных расчетах / Папилина Т.М., Прядко С.А., Фомочкина А.С. — М. РГУ нефти и газа (НИУ) имени И.М. Губкина, 2021. — 141 с.

СПАСИБО ЗА ВНИМАНИЕ!